



# CODING\_GUIDELINES.md - Development Standards & Best Practices

**GaDeMa – Game Development Management Application (v0.1 Pre-Release MVP)**  
**Status:** Production-Ready Architecture with Complete Enhancement Package



## Overview

This document defines the coding standards, naming conventions, and architectural patterns that must be followed when implementing GaDeMa v0.1. These guidelines ensure consistency, maintainability, and code quality across the entire codebase.

**Target Audience:** AI Coding Agents & Human Developers  
**Framework:** ASP.NET Core 10.0.11 + EF Core + Blazor Server



## File Structure Reference

```
src/
├── GameDev.Core/Models/           # Entity classes (ContentItem.cs, User.cs)
├── GameDev.Core/Dtos/             # API DTOs (CreateProjectDto.cs)
├── GameDev.Core/Enums/           # Type enumerations
├── GameDev.Data/                 # DbContext + migrations config
├── GameDev.Api/Controllers/       # REST endpoints (ProjectsController.cs)
├── GameDev.Api/Services/          # Business logic (ContentItemService.cs)
├── GameDev.WebApp/Pages/          # Razor pages with View Mode separation
└── GameDev.Tests/                # Unit + Integration tests
```

1

## Naming Conventions

### 1.1 Model Files & Classes

| Convention | Rule                          | Example                                  |
|------------|-------------------------------|------------------------------------------|
| File Name  | EntityName.cs with PascalCase | ContentItem.cs, User.cs, StoryOutline.cs |
| Class Name | Singular + PascalCase         | ContentItem, not ContentItems            |

|                       |                         |                                                  |
|-----------------------|-------------------------|--------------------------------------------------|
| Navigation Properties | Plural collection names | ICollection<MediaAttachment><br>MediaAttachments |
|-----------------------|-------------------------|--------------------------------------------------|

```
//  CORRECT
public class ContentItem
```

```
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public ICollection<MediaAttachment> MediaAttachments { get; set; }
}

// ❌ INCORRECT
public class contentitem // lowercase, wrong casing
{
    public GUID id // PascalCase violation
}
```

## 1.2 DTO Files & Classes

| Convention          | Rule                | Example                                |
|---------------------|---------------------|----------------------------------------|
| Creation Operations | CreateXDto suffix   | CreateProjectDto, CreateContentItemDto |
| Update Operations   | UpdateXDto suffix   | UpdateContentItemDto                   |
| Response DTOs       | ResponseXDto suffix | ContentItemResponseDto                 |

```
// ✅ CORRECT - Naming Convention Rules
public class CreateProjectDto
{
    [Required]
    public string Title { get; set; } = "";
}

public class UpdateContentItemDto
{
    [MaxLength(4096)]
    public string? Description { get; set; } = null!;
}

public class ContentItemResponseDto
{
    public Guid Id { get; set; }
    public bool Published { get; set; }
}
```

## 1.3 Controller Files & Classes

| Convention | Rule                                | Example                                             |
|------------|-------------------------------------|-----------------------------------------------------|
| File Name  | ResourceNameController.cs           | ProjectsController.cs,<br>ContentItemsController.cs |
| Class Name | Match file name + Controller suffix | Same as above                                       |

```
//  CORRECT - API Contracts naming
public class ProjectsController : ControllerBase {}
public class ContentItemsController : ControllerBase {}
```

## 1.4 Service Files & Classes

| Convention           | Rule                                             | Example                                                              |
|----------------------|--------------------------------------------------|----------------------------------------------------------------------|
| Interface Files      | <code>IXxxService.cs</code>                      | <code>IContentService.cs</code> ,<br><code>IApiAuthService.cs</code> |
| Implementation Files | <code>XxxService.cs</code> (no Interface prefix) | <code>ContentItemService.cs</code>                                   |

```
//  CORRECT - Service layer naming
public interface IContentService { }

public class ContentItemService : IContentService { }
```

## 1.5 Method Naming

| Convention    | Rule                                                       | Example                                                               |
|---------------|------------------------------------------------------------|-----------------------------------------------------------------------|
| Format        | Verb-Noun (imperative mood)                                | <code>CreateContentAsync</code> ,<br><code>GetPublishedItems()</code> |
| Async Methods | Always use <code>.Async</code> suffix for async operations | <code>ToListAsync()</code> , not <code>ToList()</code>                |
| Query Methods | Prefix with <code>Get</code> or <code>Find</code>          | <code>GetProjectsAsync()</code> ,<br><code>FindByIdAsync()</code>     |

```
//  CORRECT - Method naming conventions
public async Task<ContentItem> GetContentItemAsync(Guid id) {}
public async Task<List<StorySequence>> GetAllSequencesAsync(Guid projectId)
{}

//  INCORRECT
public async Task ContentGet(id) // No verb prefix, no Async suffix
```

## 1.6 Variable Naming

| Convention      | Rule                         | Example                                       |
|-----------------|------------------------------|-----------------------------------------------|
| Local Variables | camelCase, descriptive names | <code>healthPoints</code> not <code>hp</code> |

| Convention     | Rule                                           | Example                           |
|----------------|------------------------------------------------|-----------------------------------|
| Loop Counters  | Use single letters (i, j) when appropriate     | <code>for (int i = 0; ...)</code> |
| Private Fields | camelCase prefix with <code>_</code> if needed | <code>_userContext</code>         |

```
//  CORRECT - Variable naming
public decimal healthPoints { get; set; }
private int itemCount = 0;

//  INCORRECT - Magic numbers or unclear names
public int hp { get; set; }    // Single letter, unclear meaning
public string user { get; set; }    // Unclear, should be userId
```

## 2 Documentation Standards

### 2.1 XML Documentation Rules

| Rule                  | When to Use                                                                                              | Example                                                  |
|-----------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Complex Operations    |  Required             | Multi-step logic or non-obvious parameters               |
| Self-Explanatory Code |  Not needed           | Simple CRUD methods                                      |
| Parameters            |  All with description | <code>[Display(Name = "Title")]</code> for UI properties |

```
//  CORRECT - XML docs for complex operations
public async Task<ContentItem> CreateContentAsync(CreateContentItemDto dto)
{
    // ... implementation
}

/// <summary>
/// Creates a new content item in the database.
/// Supports polymorphic design with type-specific detail tables and view
/// mode separation.
/// </summary>
/// <param name="dto">DTO object with required fields for creation</param>
/// <returns>The newly created content item instance</returns>
/// <exception cref="ValidationException">Thrown when DTO validation
/// fails</exception>
public async Task<ContentItem> CreateContentAsync(CreateContentItemDto dto)
{
    // ... implementation
}
```

### 2.2 XML Documentation Template

```
using System.ComponentModel.DataAnnotations;

/// <summary>
/// Represents a content item in the game development management system.
/// Supports polymorphic design with type-specific detail tables and view
/// mode separation.
/// </summary>
public class ContentItem
{
    /// <summary>
    /// Unique identifier for the content item.
    /// </summary>
    [Key]
    public Guid Id { get; set; } = Guid.NewGuid();

    /// <summary>
    /// Project ID that this content item belongs to.
    /// </summary>
    [Required]
    public Guid ProjectId { get; set; }

    /// <summary>
    /// Content type (e.g., Character, World, DialogueNode).
    /// </summary>
    [EnumDataType(typeof(ContentTypeEnum)), Required]
    public ContentTypeEnum ContentType { get; set; }

    // ... other properties
}
```

## 2.3 Labels for UI Properties (continued)

| Convention               | Rule                                           | Example                   |
|--------------------------|------------------------------------------------|---------------------------|
| <b>Display Attribute</b> | [Display(Name = "...")] for all DTO properties | [Display(Name = "Title")] |
| <b>Max Length</b>        | Add for string fields in DTOs                  | [MaxLength(128)]          |

```
// ✓ CORRECT - DTO property labels
public class CreateContentItemDto
{
    [Required]
    [Display(Name = "Project ID")]
    public Guid ProjectId { get; set; }

    [EnumDataType(typeof(ContentTypeEnum)), Required]
    [Display(Name = "Content Type")]
    public ContentTypeEnum ContentType { get; set; }
```

```

    [MaxLength(128), Required]
    [Display(Name = "Title")]
    public string Title { get; set; } = "";

    // ... other properties
}
```

2.4 Validation Attributes Placement

| Rule                  | When to Use                                                                                                                | Example                                        |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| <b>DTO Properties</b> |  All validation on DTOs, NOT models       | CreateProjectDto.cs has [Required] attributes  |
| <b>Model Classes</b>  |  No validation attributes (use DbContext) | Entity classes use Fluent API constraints only |

```

//  CORRECT - Validation on DTOs only
public class CreateProjectDto
{
    [Required]
    public string Title { get; set; } = "";
}

//  INCORRECT - Don't add validation to models
public class ContentItem
{
    // NO VALIDATION ATTRIBUTES HERE
    public string Description { get; set; } = "";
}
```

3 Database & Performance Guidelines

3.1 Eager Loading with Include()

| Rule                      | When to Use                                                                                                                  | Example                                                              |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| <b>Always Use Include</b> |  All relationship queries                 | _context.ContentItems.Include(i => i.MediaAttachments).ToListAsync() |
| <b>Avoid N+1 Queries</b>  |  Never use Where on navigation properties | Don't query attachments separately in a loop                         |

```

//  CORRECT - Eager loading to avoid N+1 queries
var contentItem = await _context.ContentItems
    .Include(ci => ci.MediaAttachments)
```

```

        .Include(ci => ci.ContentTags)
        .ThenInclude(ct => ct.Tag) // Include junction + tag data
        .FirstOrDefaultAsync(ci => ci.Id == id);

// ❌ INCORRECT - N+1 problem
var contentItems = await _context.ContentItems.ToListAsync();
foreach (var item in contentItems)
{
    var attachments = await _context.MediaAttachments
        .Where(a => a.ContentItemId == item.Id).ToListAsync(); // N+1!
}

```

## 3.2 Pagination Best Practices

| Rule                     | When to Use                           | Example                       |
|--------------------------|---------------------------------------|-------------------------------|
| <b>Default Page Size</b> | ✅ Always use pagination (default: 20) | <code>Skip(0).Take(20)</code> |
| <b>Max Page Size</b>     | ❌ Never allow > 100 items per page    | Validate pageSize parameter   |

```

// ✅ CORRECT - Paginated query with default size
public async Task<PaginationResult<ContentItemDto>>
GetContentItemsAsync(Guid projectId, int page = 1, int pageSize = 20)
{
    return new PaginationResult<ContentItemDto>
    {
        Data = await _context.ContentItems
            .Where(c => c.ProjectId == projectId)
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .Select(c => new ContentItemDto { ... })
            .ToListAsync(),
        TotalItems = await _context.ContentItems.CountAsync(c =>
c.ProjectId == projectId),
        CurrentPage = page,
        PageSize = pageSize,
        TotalPages = (int)Math.Ceiling(TotalItems / (double)pageSize)
    };
}

```

## 3.3 Index Configuration

| Rule                               | When to Use                       | Example                                          |
|------------------------------------|-----------------------------------|--------------------------------------------------|
| <b>Frequently Filtered Columns</b> | ✅ Add indexes on filtered columns | <code>HasIndex(e =&gt; e.ContentType)</code>     |
| <b>Unique Constraints</b>          | ✅ Ensure slugs are unique         | <code>HasIndex(e =&gt; e.Slug).IsUnique()</code> |

```

modelBuilder.Entity<ContentItem>(entity =>
{
    entity.HasKey(e => e.Id);

    // Indexes for frequently filtered columns
    entity.HasIndex(e => e.ContentType);
    entity.HasIndex(e => e.Status);
    entity.HasIndex(e => e.Published);

    // Unique slug index
    entity.HasIndex(e => e.Slug).IsUnique();
});

```

3.4 Query Performance Guidelines

| Rule                                   | When to Use                                      | Example                                                    |
|----------------------------------------|--------------------------------------------------|------------------------------------------------------------|
| Avoid Complex OR Conditions            | <div> ✔ Limit complexity in WHERE clauses </div> | Keep under 5 conditions per query                          |
| Use <b>Where</b> Before <b>Include</b> | <div> ✔ Filter first, then eager load </div>     | <code>_context.ContentItems.Where(...).Include(...)</code> |

```

// ✔ CORRECT - Filter before including relationships
var items = await _context.ContentItems
    .Where(c => c.Published && c.ContentType == ContentTypeEnum.Character)
    .Include(c => c.MediaAttachments)
    .ThenInclude(m => m.ContentItem)
    .ToListAsync();

// ✖ INCORRECT - Include before filtering (wasted queries)
var items = await _context.ContentItems
    .Include(c => c.MediaAttachments)
    .ThenInclude(m => m.ContentItem)
    .Where(c => c.Published && c.ContentType == ContentTypeEnum.Character)
    .ToListAsync(); // Includes relationships for all items first

```

4 DTO Design Patterns

4.1 Naming Convention Rules (See Section 1.2)

| Rule                | When to Use                            | Example                                             |
|---------------------|----------------------------------------|-----------------------------------------------------|
| Creation Operations | <div> ✔ Use <b>CreateXDto</b> </div>   | <code>CreateProjectDto, CreateContentItemDto</code> |
| Update Operations   | <div> ✔ Use <b>UpdateXDto</b> </div>   | <code>UpdateContentItemDto</code>                   |
| Response DTOs       | <div> ✔ Use <b>ResponseXDto</b> </div> | <code>ContentItemResponseDto</code>                 |

### 4.2 Required vs Optional Field Patterns (See Section 1.2)

```
// ✅ CORRECT - Required fields use [Required] attribute
[Required] public string Title { get; set; } = "";

// ❌ INCORRECT - Don't make strings nullable unless needed
public string? Description { get; set; } = null!; // Should be: = ""

// ✅ CORRECT - Collections initialized with empty list
public ICollection<Guid> TagIds { get; set; } = new List<Guid>();

// ❌ INCORRECT - Don't leave collections as null
public ICollection<string>? Tags { get; set; } // Should initialize
```

### 4.3 Validation Attributes Placement (See Section 2.4)

| Rule      | When to Use                 | Example                       |
|-----------|-----------------------------|-------------------------------|
| MaxLength | ✅ For string fields in DTOs | [MaxLength(128)] for title    |
| Range     | ✅ For numeric fields        | [Range(1, 100)] for page size |

```
// ✅ CORRECT - Validation attributes on DTO properties
[Required]
[Display(Name = "Page Size")]
public int PageSize { get; set; } = 20;

// ❌ INCORRECT - Don't validate in controller actions
var items = await
_context.ContentItems.Skip(0).Take(pageSize).ToListAsync(); // No
validation!
```

### 4.4 DTO File Organization

| Convention         | Rule                                 | Example                                            |
|--------------------|--------------------------------------|----------------------------------------------------|
| Folder Structure   | Separate Dtos folder with subfolders | GameDev.Core/Dtos/Projects/CreateProjectDto.cs     |
| Namespace Mappings | Follow folder path patterns          | src/GameDev.Core/Dtos → GameDev.Core.Dtos.Projects |

```
# ✅ CORRECT - Folder structure
GameDev.Core/Dtos/
├── Projects/
│   └── CreateProjectDto.cs
```

```
|   └─ UpdateProjectDto.cs
|   └─ ContentItems/
|       └─ CreateContentItemDto.cs
|       └─ UpdateContentItemDto.cs
|   └─ Common/
|       └─ PaginationResult.cs
|       └─ ErrorResponseDto.cs
```

## 5 Error Handling & Exception Standards (See Section 8 in specification)

### 5.1 Standard Exception Types

| Rule                  | When to Use                  | Example                                                |
|-----------------------|------------------------------|--------------------------------------------------------|
| NotFoundException     | ✔ Resource not found         | throw new NotFoundException("Character not found", id) |
| ValidationException   | ✔ Input validation failed    | throw new ValidationException(errors)                  |
| UnauthorizedException | ✔ Missing/invalid auth token | throw new UnauthorizedException()                      |

```
// ✔ CORRECT - Standard exception types
public class NotFoundException : Exception {
    public NotFoundException(string entity, Guid id)
        : base($"{entity} with ID {id} not found") {}
}

public class ValidationException : Exception {
    public string[] Errors { get; }

    public ValidationException(IEnumerable<string> errors)
    {
        Errors = errors.ToArray();
    }
}
```

### 5.2 Error Response Structure (See Section 8 in specification)

| Rule              | When to Use                       | Example                                                  |
|-------------------|-----------------------------------|----------------------------------------------------------|
| 4xx Status Codes  | ✔ Standard error response         | { "success": false, "errors": [...] }                    |
| 500 Server Errors | ⚠ Log internally, generic message | { "success": false, "message": "Internal server error" } |

```
// ✅ CORRECT - Error response format
BadRequest(new {
    success = false,
    errors = ["Description cannot be empty", "Title is required"]
});

// ✅ CORRECT - Validation exception with multiple errors
var validationErrors = new List<string> {
    "Description cannot be empty",
    "Title is required"
};
throw new ValidationException(validationErrors);
```

5.3 Controller Error Handling Pattern

| Rule                           | When to Use                    | Example                                  |
|--------------------------------|--------------------------------|------------------------------------------|
| Always Check ModelState        | ✅ Before processing logic      | ModelState.IsValid in controller actions |
| Never Throw Generic Exceptions | ❌ Use specific exception types | Not throw new Exception()                |

```
// ✅ CORRECT - Controller error handling pattern
[HttpPost]
public async Task<IActionResult> CreateContentAsync([FromBody] CreateContentItemDto dto)
{
    // Always check validation first
    if (!ModelState.IsValid)
    {
        var errors = ModelState.Values
            .SelectMany(v => v.Errors)
            .Select(e => e.ErrorMessage);
        return BadRequest(new { success = false, errors });
    }

    try
    {
        // Process creation logic
        var item = await _contentService.CreateContentAsync(dto, Guid.NewGuid());
        return Ok(item);
    }
    catch (NotFoundException ex) when (ex.InnerException != null)
    {
        // Handle related entity not found
        return NotFound(ex.Message);
    }
}
```

```
    }  
}
```

## 6 Security Implementation Details (See Section 9 in specification)

### 6.1 Password Hashing Rules

| Rule                   | When to Use                                | Example                     |
|------------------------|--------------------------------------------|-----------------------------|
| BCrypt with Salt       | ✔ Always hash passwords before storage     | HashPasswordAsync(password) |
| Never Store Plain Text | ✘ Never commit plain text passwords to git | Don't store in database!    |

```
// ✔ CORRECT - Password hashing implementation  
public async Task<User> HashPasswordAsync(string password)  
{  
    var options = new Rfc2898DeriveBytes(password, "salt", 1000,  
HashAlgorithmName.HmacSha512);  
    return _context.Users.Add(new User  
    {  
        PasswordHash = Convert.ToBase64String(options.GetBytes(32)),  
        // ... other fields  
    });  
}
```

### 6.2 API Token Security

| Rule                   | When to Use                        | Example                                     |
|------------------------|------------------------------------|---------------------------------------------|
| SHA256 + Salt          | ✔ Hash tokens before storage       | TokenHash =<br>Convert.ToBase64String(hash) |
| Never Store Plain Text | ✘ Never commit token hashes to git | Don't expose in logs!                       |

```
// ✔ CORRECT - API token hashing implementation  
public async Task<ProjectToken> CreateApiTokenAsync(Guid projectId, string  
plainTextToken)  
{  
    var salt = GenerateUniqueSalt(); // Random 32-byte salt  
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes(plainTextToken +  
salt));  
  
    return new ProjectToken  
    {  

```

```
        ProjectId = projectId,
        TokenHash = Convert.ToBase64String(hash),
        Salt = Convert.ToBase64String(salt)
    };
}
```

6.3 File Upload Security

| Rule                  | When to Use                  | Example                    |
|-----------------------|------------------------------|----------------------------|
| MIME Type Validation  | ✔ Validate before processing | Check against allowed list |
| File Size Limit       | ✔ Enforce max size           | Max 100MB per file         |
| Filename Sanitization | ✔ Remove path separators     | Use UUID-based names       |

```
// ✔ CORRECT - File upload security
public const int MaxFileSize = 100 * 1024 * 1024; // 100MB in bytes
public static readonly string[] AllowedMimeTypes =
{
    "image/png", "image/jpeg", "application/pdf", "text/plain"
};

public async Task<IActionResult> UploadFileAsync(IFormFile file)
{
    // Validate MIME type
    if (!AllowedMimeTypes.Contains(file.ContentType))
        return BadRequest("Invalid file type. Only images and PDFs
allowed.");

    // Validate file size
    if (file.Length > MaxFileSize)
        return BadRequest($"File too large. Maximum size: {MaxFileSize /
1024 / 1024}MB");

    // Sanitize filename and upload to storage
    var safeName = Path.GetFileNameWithoutExtension(file.FileName);
    var extension = Path.GetExtension(file.FileName);
    var uniqueName = $"content-
{Guid.NewGuid()}_{DateTime.UtcNow:yyyyMMddHHmmss}{extension}";

    await using var stream = new FileStream($"uploads/{uniqueName}",
FileMode.Create);
    await file.CopyToAsync(stream);

    return Ok(new { filename = safeName, storagePath =
$"/uploads/{uniqueName}" });
}
```

6.4 Input Sanitization

| Rule                        | When to Use              | Example                           |
|-----------------------------|--------------------------|-----------------------------------|
| <b>HTML Escaping</b>        | ✅ For Description fields | <code>HtmlEncoder.Encode()</code> |
| <b>Never Store Raw HTML</b> | ❌ Escape before saving   | Prevent XSS attacks               |

```
// ✅ CORRECT - HTML sanitization to prevent XSS
public string SanitizeHtml(string htmlContent)
{
    return new HtmlEncoder().Encode(htmlContent);
}
```

## 6.5 JWT Token Configuration (See Section 9 in specification)

| Rule                    | When to Use                | Example                                 |
|-------------------------|----------------------------|-----------------------------------------|
| <b>Token Expiration</b> | ✅ Set appropriate expiry   | 1 hour for access tokens                |
| <b>Clock Skew</b>       | ⚠️ Minimal clock tolerance | <code>TimeSpan.Zero</code> for security |

```
// ✅ CORRECT - JWT configuration with expiration
services.AddAuthentication()
    .AddJwtBearer(options =>
    {
        options.RequireHttpsMetadata = false;
        options.SaveToken = true;
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(key)),
            ValidateIssuer = false,
            ValidateAudience = false,
            ClockSkew = TimeSpan.Zero // Minimal clock tolerance
        };
    });
```

## 7 Performance Guidelines (See Section 9 in specification)

### 7.1 Query Optimization Rules

| Rule                           | When to Use                    | Example                                           |
|--------------------------------|--------------------------------|---------------------------------------------------|
| <b>Always Use Include</b>      | ✅ For all relationship queries | <code>.Include(i =&gt; i.MediaAttachments)</code> |
| <b>Paginate List Endpoints</b> | ✅ Default: 20 items, max: 100  | <code>Skip().Take()</code> for pagination         |

7.2 Caching Strategy (See Section 9 in specification)

```
//  CORRECT - Redis caching strategy
public async Task<List<StorySequence>> GetSequencesWithCacheAsync(Guid
projectId)
{
    var cacheKey = $"sequences:{projectId}";
    return await _redisCache.GetOrCreateAsync(cacheKey, async entry =>
    {
        // Set appropriate expiration (5 min for drafts, 1 hour for
published)
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(1);

        var sequences = await _context.StorySequences
            .Where(s => s.ProjectId == projectId)
            .Include(s => s.OutlineSummary)
            .ToListAsync();

        return sequences;
    });
}
```

7.3 File Upload Optimization (See Section 9 in specification)

| Rule                      | When to Use                                                                                                                | Example                            |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Stream Large Files        |  Use <code>FileStream</code> not memory | Don't load entire file into memory |
| Validate Before Uploading |  Check MIME type and size first         | Prevents OOM exceptions            |

```
//  CORRECT - File stream for large uploads
public async Task UploadLargeFileAsync(Guid contentItemId, string fileName)
{
    var filePath = $"uploads/{Guid.NewGuid()}_{fileName}";

    await using var fileStream = new FileStream(filePath, FileMode.Create);
    // Stream from source instead of loading entire file into memory
    await stream.CopyToAsync(fileStream);
}
```

 UI/UX & ADHD-Friendly Design Guidelines

8.1 View Mode Separation (See Section 4 in specification)

| Rule                | When to Use                                                                                                         | Example                           |
|---------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| PrivateWriting Mode |  Admin interface with full tools | Show all fields + version history |

| Rule              | When to Use              | Example                                 |
|-------------------|--------------------------|-----------------------------------------|
| Presentation Mode | ✔ Clean public view only | Show published fields, no admin details |

```
// ✔ CORRECT - View mode separation in controller
[HttpGet("{id}")]
public async Task<IActionResult> GetContentItemAsync(Guid id, [FromQuery]
ViewModelEnum viewMode = ViewModelEnum.PrivateWriting)
{
    var item = await _context.ContentItems.FindAsync(id);

    // PrivateWriting mode: return full content + admin tools
    if (viewMode == ViewModelEnum.PrivateWriting)
    {
        return Ok(item); // Includes all fields + version history
    }

    // Presentation mode: return only published fields + clean layout
    return Ok(new
    {
        id = item.Id,
        title = item.Title,
        description = item.Description,
        published = item.Published
    });
}
```

8.2 Focus Mode UI (See Section 4 in specification)

| Rule                    | When to Use                            | Example                |
|-------------------------|----------------------------------------|------------------------|
| Single-Task View        | ✔ Hide sidebars, show only active task | Reduces cognitive load |
| Quick Wins Highlighting | ✔ Green badge for easy tasks           | ADHD-friendly momentum |

```
// ✔ CORRECT - Focus mode UI pattern (Blazor example)
@if (model.ViewMode == ViewModelEnum.PrivateWriting)
{
    <div class="admin-panel">
        <button onclick="viewModelService.AddEditButton()">✎ Edit
Content</button>
        <button onclick="viewModelService.AddVersionControl()">📄 Version
History</button>
        <button onclick="viewModelService.AddTaskManagement()">✔ Manage
Tasks</button>
    </div>
}
```

8.3 Task Difficulty Filtering (See Section 4 in specification)

| Rule                 | When to Use                             | Example                                    |
|----------------------|-----------------------------------------|--------------------------------------------|
| ADHD-Friendly Filter | ✔️ Prioritize Easy/QuickWin tasks first | <code>OrderBy(t =&gt; t.Difficulty)</code> |
| Estimated Minutes    | ✔️ Show short tasks at top of list      | Reduces overwhelm                          |

```
// ✔️ CORRECT - ADHD-friendly task filtering
public async Task<List<Task>> GetQuickWinTasksAsync(Guid projectId)
{
    return await _context.Tasks
        .Where(t => t.ProjectId == projectId && t.IsQuickWin)
        .Include(t => t.Comments)
        .OrderBy(t => t.EstimatedMinutes) // Show shortest tasks first
        .ToListAsync();
}
```

## 9 Testing Requirements (See Section 13 in specification)

### 9.1 Unit Tests Coverage Target

| Rule                   | When to Use                | Example                               |
|------------------------|----------------------------|---------------------------------------|
| Coverage Target        | ✔️ ≥80% code coverage      | Business logic only                   |
| Mock External Services | ✔️ Mock AI, Email in tests | Don't test external services directly |

```
// ✔️ CORRECT - Unit test pattern with ≥80% coverage target
public class ContentItemServiceTests : IDisposable
{
    private readonly GameDbContext _context;

    [Fact]
    public async Task
    CreateContentAsync_WhenDescriptionIsNull_ShouldThrowValidationException()
    {
        // Arrange: Setup test data with null description
        var contentDto = new ContentItemDto
        {
            Title = "Test Character",
            Description = null, // Invalid case
            ContentType = ContentTypeEnum.Character
        };

        // Act: Execute service method
        var exception = await Assert.ThrowsAsync<ValidationException>(
            async () => await
            _contentService.CreateContentAsync(contentDto, Guid.NewGuid()));

        // Assert: Verify error message
    }
}
```

```
        exception.Message.Should().Contain("Description cannot be empty");
    }
}
```

9.2 Integration Tests Pattern (See Section 13 in specification)

```
// ✓ CORRECT - Integration test structure
[Fact]
public async Task GetPublishedContentAsync_ReturnsOnlyPublishedItems()
{
    var client = new WebApplicationFactory<GameDbContext>().CreateClient();

    // Act: Call API endpoint
    var response = await client.GetAsync("/api/v1/content/items?
projectId=abc&published=true");

    // Assert: Verify published items only
    response.StatusCode.Should().Be(HttpStatusCode.OK);
    var content = await
response.Content.ReadFromJsonAsync<List<ContentItemDto>>();
    content.All(c => c.Published).Should().BeTrue();
}
```

9.3 Git Commit Guidelines (See Section 17 in specification)

| Rule          | When to Use                | Example                              |
|---------------|----------------------------|--------------------------------------|
| Format        | ✓ <Type>: <Subject>        | feat: add external reference support |
| Branch Naming | ✓ Descriptive branch names | feature/external-references          |

```
# ✓ CORRECT - Git commit message format
git commit -m "feat: add external reference support for Pinterest boards"
git commit -m "fix: resolve pagination issue in task list endpoint"
git commit -m "docs: update README with v0.1 features and deployment
instructions"

# ✓ CORRECT - Branch naming convention
git checkout -b feature/external-references
git checkout -b fix/view-mode-separation-bug
git checkout -b chore/update-gitignore
```

10 Anti-Patterns to Avoid (See Section 4 in specification)

10.1 No Repository Pattern

| Rule                               | Why?                                                                                                                   | Example                   |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------|
| <b>Avoid Repository Interfaces</b> |  Adds unnecessary abstraction for MVP | Use direct EF Core access |

```
//  CORRECT - Direct EF Core access (No Repository Pattern)
public class ContentItemService
{
    private readonly GameDbContext _context;

    public ContentItemService(GameDbContext context)
    {
        _context = context; // Direct DbContext dependency
    }

    public async Task<ContentItem> GetContentItemAsync(Guid id)
    {
        return await _context.ContentItems.FindAsync(id); // Simple and
clear
    }
}

//  INCORRECT - Repository Pattern (unnecessary abstraction for MVP)
public interface IRepository<T> where T : class {}
public class ContentItemRepository : IRepository<ContentItem> {}
```

## 10.2 No Hierarchical Tasks

| Rule                       | Why?                                                                                                         | Example                |
|----------------------------|--------------------------------------------------------------------------------------------------------------|------------------------|
| <b>Flat Task Structure</b> |  ADHD-friendly simplicity | No complex hierarchies |

```
//  CORRECT - Flat task structure with tags
public class Task
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid ProjectId { get; set; }
    public int Difficulty { get; set; } // Easy, Medium, Hard
    public bool IsQuickWin { get; set; } // ADHD-friendly feature
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

//  INCORRECT - Hierarchical tasks (complex epics/stories)
public class Epic { }
public class Story { }
public class TaskItem { }
```

## 10.3 No Over-Documentation

| Rule                     | Why?                    | Example          |
|--------------------------|-------------------------|------------------|
| <b>Simple Operations</b> | ✅ Self-explanatory code | Minimal XML docs |

```
// ✅ CORRECT - Simple CRUD methods without XML docs
public async Task<ContentItem> CreateContentAsync(CreateContentItemDto dto,
Guid projectId)
{
    var item = new ContentItem
    {
        ProjectId = projectId,
        ContentType = dto.ContentType,
        Title = dto.Title,
        Description = dto.Description
    };

    await _context.ContentItems.AddAsync(item);
    return item;
}

// ❌ INCORRECT - Over-documentation for simple operations
/// <summary>Creates a content item...</summary> /// <param name="dto">...
</param>
public async Task<ContentItem> CreateContentAsync(CreateContentItemDto dto,
Guid projectId) {} // Too much!
```

## 10.4 No Magic Numbers

| Rule                       | Why?                              | Example                    |
|----------------------------|-----------------------------------|----------------------------|
| <b>Use Enums/Constants</b> | ✅ Maintainability and readability | Don't use hardcoded values |

```
// ✅ CORRECT - Using enums instead of magic numbers
public class Task
{
    public int Difficulty { get; set; } // Enum: Easy(0), Medium(1),
Hard(2)
    public bool IsQuickWin { get; set; }
}

// ❌ INCORRECT - Magic numbers in code
if (task.Difficulty == 1) // What does 1 mean?
{
    // This is bad! Use Enum.Parse<TaskDifficulty>(...) or constants
    instead
}
}
```

## 1.1 Code Review Checklist

| Aspect        | Question to Ask Before Committing                                             | Expected Answer                            |
|---------------|-------------------------------------------------------------------------------|--------------------------------------------|
| Naming        | Is this following PascalCase for models and Verb-Noun for methods?            | ✓ Yes                                      |
| Documentation | Does this need XML docs, or is the code self-explanatory?                     | ✓ Minimal XML docs only                    |
| Validation    | Are all input fields validated with attributes on DTOs?                       | ✓ All DTOs have [Required] and [MaxLength] |
| Security      | Are passwords hashed, tokens encrypted, and inputs sanitized?                 | ✓ Yes - BCrypt, SHA256, HtmlEncoder        |
| Performance   | Am I using <code>Include()</code> for eager loading and avoiding N+1 queries? | ✓ All relationship queries use Include()   |

## 1.2 Summary: Key Rules Checklist

| Category              | Rule                                                | Must Implement? | Priority |
|-----------------------|-----------------------------------------------------|-----------------|----------|
| Naming                | PascalCase models, Verb-Noun methods                | ✓ Yes           | High     |
| DTOs                  | CreateXDto/UpdateXDto naming, validation attributes | ✓ Yes           | High     |
| Database              | Eager loading with Include(), pagination            | ✓ Yes           | High     |
| Security              | Password hashing (BCrypt), token hashing (SHA256)   | ✓ Yes           | Critical |
| File Uploads          | MIME type validation, file size limits              | ✓ Yes           | High     |
| View Modes            | PrivateWriting vs Presentation separation           | ✓ Yes           | High     |
| No Repository Pattern | Direct EF Core access only                          | ✓ Yes           | Medium   |
| Anti-Patterns         | No magic numbers, minimal XML docs                  | ✓ Yes           | Medium   |